

AlgoList 프로젝트 설계 문서

문서 정보

문서명	AlgoList 프로젝트 설계 문서
작성일	2026-06-25
제출 범위	요구사항 정의서, Use-Case 다이어그램, 클래스 다이어그램, ER 다이어그램, 간트 차트, 화면 설계서

1. 요구사항 정의서

1.1 기능 요구사항 (Must Have)

분류	기능 코드	기능명	설명
인증	AUTH-001	로그인	사용자 인증 기능
인증	AUTH-002	로그아웃	사용자 로그아웃 기능
회원 관리	MEM-001	회원가입	새로운 사용자 등록
회원 관리	MEM-002	회원정보 수정	사용자 정보 변경 (이름, 비밀번호 등)
회원 관리	MEM-003	회원정보 조회	관리자 페이지에서 회원 목록 조회
회원 관리	MEM-004	회원 탈퇴	사용자 계정 삭제
문제 관리	PROB-001	문제 통합 검색	외부 API 요청/크롤링을 통해 여러 사이트의 문제를 통합 검색
문제 관리	PROB-002	내 문제 조회	사용자가 추가한 문제 목록 조회 및 검색
문제 관리	PROB-003	내 문제 추가	검색한 문제를 내 문제 목록에 추가
문제 관리	PROB-004	내 문제 삭제	내 문제 목록에서 문제 제거
소스코드 관리	SRC-001	소스코드 추가	파일 업로드 또는 텍스트 입력으로 소스코드 등록
소스코드 관리	SRC-002	소스코드 조회	저장된 소스코드 조회
소스코드 관리	SRC-003	소스코드 수정	기존 소스코드 내용 수정
소스코드 관리	SRC-004	소스코드 삭제	저장된 소스코드 제거
오답노트 관리	NOTE-001	오답노트 추가	문제에 대한 오답노트 작성
오답노트 관리	NOTE-002	오답노트 조회	작성된 오답노트 조회

분류	기능 코드	기능명	설명
오답노트 관리	NOTE-003	오답노트 수정	오답노트 내용 수정
오답노트 관리	NOTE-004	오답노트 삭제	오답노트 제거
복습 주기 관리	REV-001	문제 등급 설정/수정	문제 등급 설정 (등급에 기본값 부여, 수정만 구현)
복습 주기 관리	REV-002	복습 리마인드	등급별 복습 일자 자동 계산 및 알림
깃허브 자동 푸시	GIT-001	깃허브 계정 연동	GitHub OAuth를 통한 사용자 계정 연동
깃허브 자동 푸시	GIT-002	대상 레포지토리 설정	소스코드를 푸시할 레포지토리 선택
깃허브 자동 푸시	GIT-003	소스코드 자동 푸시	소스코드 업로드 시 연결된 레포지토리에 자동 커밋 & 푸시

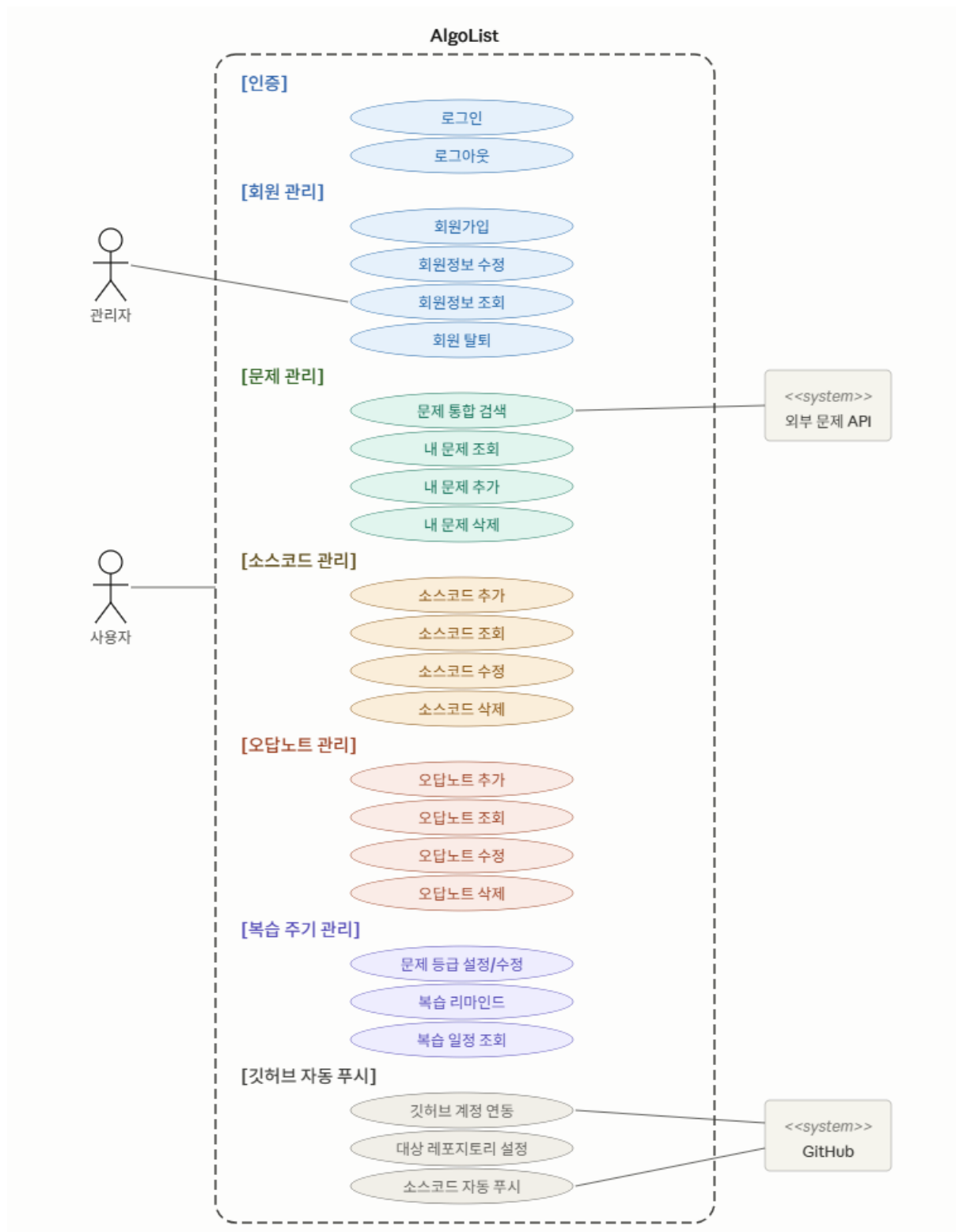
1.2 비기능 요구사항

#	항목	설명
1	검색 횟수 제한	분당 요청 횟수를 제한하여 서비스의 API 요청이 차단당하는 것을 방지

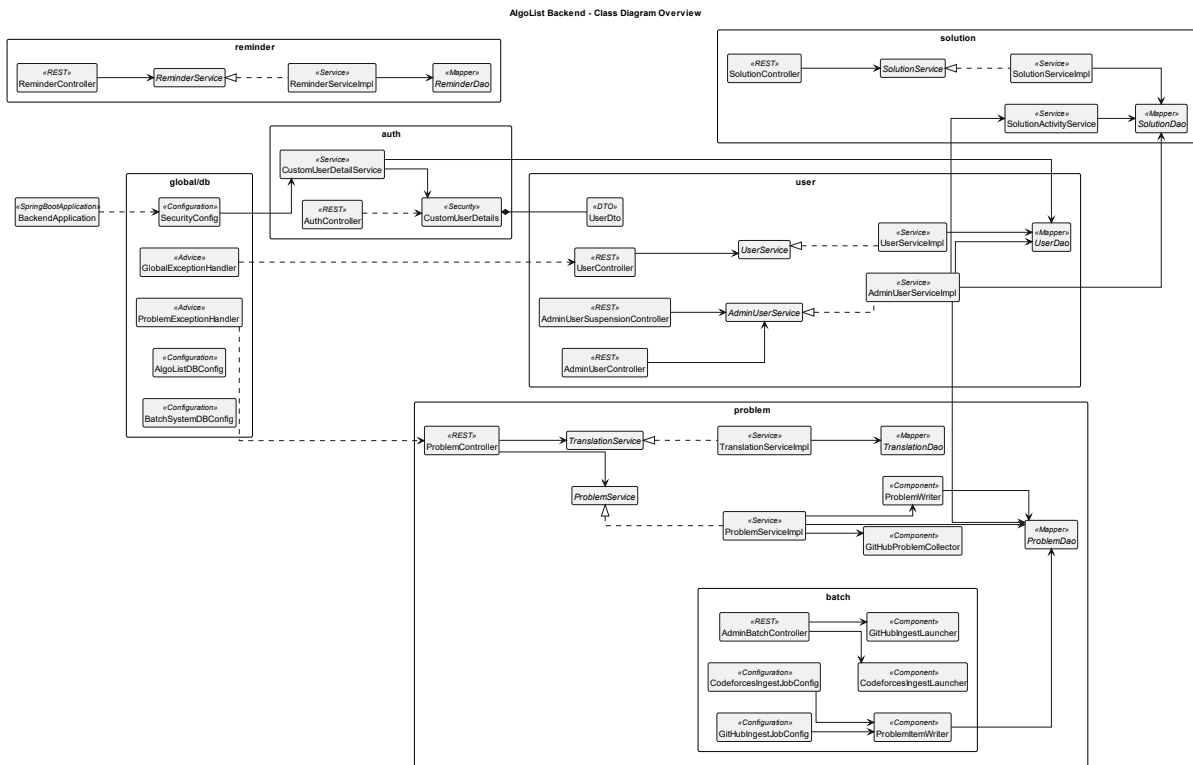
1.3 부가 기능 (Should Have)

#	기능명	설명
1	AI 소스코드 제공	AI를 통한 정답 소스코드 제공
2	검색 기록 캐싱	검색 결과를 캐싱하여 응답 속도 향상
3	회원 등급제	사용자별 소스코드 저장 용량 제한, 결제 시 더 많은 소스코드 저장 가능

2. Use-Case 다이어그램

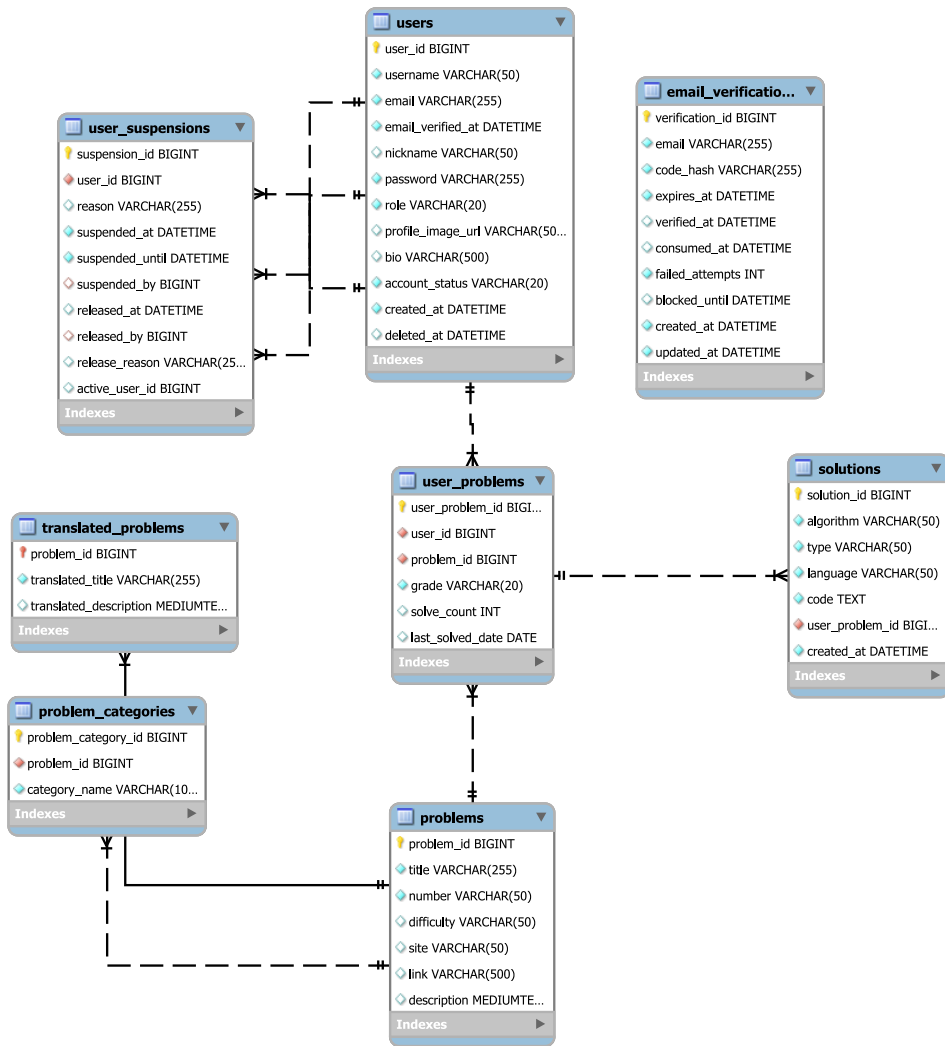


3. 클래스 다이어그램



4. ER 다이어그램

4.1 AlgoList 데이터베이스



4.2 batch_system 추가 테이블

failed_proble...
id BIGINT
site VARCHAR(50)
number VARCHAR(50)
file_url VARCHAR(1000)
reason VARCHAR(1000)
status VARCHAR(20)
created_at DATETIME
Indexes

failed_pages
id BIGINT
source VARCHAR(50)
page_offset INT
page_size INT
reason VARCHAR(50...)
status VARCHAR(20)
created_at DATETIME
Indexes

5. WBS & 간트 차트

색상 막대는 작업 기간을 의미하며, 옅은 붉은 배경은 주말입니다.

[illegible]

[illegible]

S [4014] 활주로 건설

→ 제목 (클릭하면 사이트로 이동)

가장 최근 푼 날짜: 2026-03-09

문제 등급: ● RED (2026-03-10) → 복습해야 하는 날짜

▶ 알고리즘 분류 → toggle 방식으로 원한 때만 보도록 함.

푼 횟수: 4

Solution_4014_오양호.java	✓	↗
Solution_4014_오양호_복습1.java	✓	↗
Solution_4014_오양호_복습2.java	✓	↗
Solution_4014_오양호_다른풀이1.java	✗	↗

깃허브에 올렸는지

→ 깃허브로 커밋 푸시

문제 / 분류 / git

검색창



→ 리스트 내 문제 검색



→ 다중 삭제

B [17471] 거리멘터링



→ 문제 추가

S [4014] 활주로 건설



→ 수정, 개별 삭제

→ 선택된 요소를 관리할 수 있게
강조 효과.

요소 선택은 반응형 변수

"SelectedItem"에서 관리

S [4014] 활주로 건설

→ 제목 (클릭하면 사이트로 이동)

가장 최근 푼 날짜: 2026-03-09

문제 등급: ● RED (2026-03-10) → 복습해야 하는 날짜

▶ 알고리즘 분류 → toggle 방식으로 원한 때만 보도록 함.

푼 횟수: 4

Solution_4014_오양호.java	✓ ↗
Solution_4014_오양호_복습1.java	✓ ↗
Solution_4014_오양호_복습2.java	✓ ↗
Solution_4014_오양호_다른풀이1.java	✗ ↗

깃허브에 올렸는지

→ 깃허브에 커밋 표시

문제 분류 히트

검색창



→ 리스트 내 문제 검색



→ 다중 삭제

BOJ

[17471] 계리멘터링



→ 문제 추가

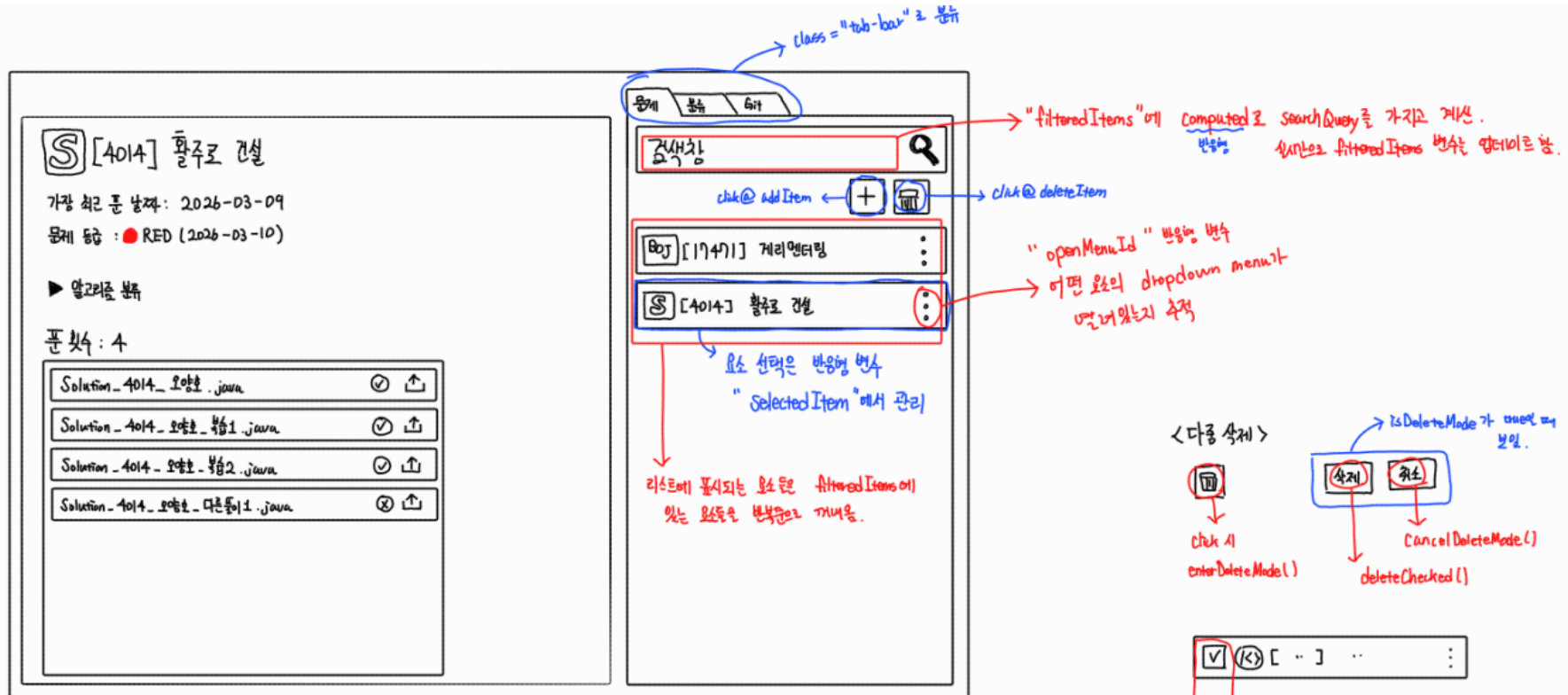
S

[4014] 활주로 건설



→ 수정, 개별 삭제

→ 선택된 요소를 인지할 수 있게 강조 효과.



TODO

- ① "문제" 요소에 필요한 정보 구체화 하기
- ② 데이터 추가 로직 구체화

1차 Mockup 목표

- ① 외부 API에서 데이터를 받아서 "문제" 요소의 형태로 채우고 리스트에 추가할 수 있도록.
 Mock up 단계에서는 backend에서 데이터를 받는 부분을 고려해서 하고,
 백과 연결 끝에는 dummy data로 채워서 동작만 확인한다.

- ② 1차 Mockup에서는

"문제 검색" → "요소 추가" → "요소 선택" → "컨텐츠 확인" → "요소 삭제" 까지만.

설명: DetailSection.vue 의 하위 컴포넌트로, 사용자의 소스코드를 관리한다.

▼ First Trial

☐

Main_17471-DYH.java

▼ Recapitulate

☐

Main_17471-DYH-Recap01.java

☐

Main_17471-DYH-Recap02.java

▼ Optimized

☐

Main_17471-DYH-Opt01.java

← 생김새는 이런 모양인데...

구조에 대해서는 조금 더 생각해봐야 할듯...

[사이트] - [문제 번호] - [이름] - [사용 약어증] - [최적화 or 단순화] - [확장자명]

→ 이게 제법 나은듯...?